

φ -Calculus: Object-Oriented Formalism

Ver: 0.0.0

YEGOR BUGAYENKO, Huawei, Russia

MAXIM TRUNNIKOV, Huawei, Russia

Object-oriented programming (OOP) is one of the most popular paradigms used for building software systems¹. However, despite its industrial and academic popularity, OOP is still missing a formal apparatus similar to λ -calculus, which functional programming is based on. A number of attempts were made to formalize OOP, but none of them managed to cover all the features available in modern OO programming languages, such as C++ or Java. We have made yet another attempt and created φ -calculus. This paper does not demonstrate the practical use or effect of φ -calculus but merely explains it.

Additional Key Words and Phrases: Object-Oriented Programming, Object Calculus

1 Introduction

Object-oriented programming has become the dominant paradigm for software development, with languages such as Java, C++, Python, and C# being among the most widely used in industry [? ? ? ? ? ?]. Despite this prevalence, the theoretical foundations of OOP remain less developed than those of functional programming [? ?], which has long benefited from the λ -calculus [? ?] as a formal basis. The absence of a comparable formalism for OOP hinders rigorous reasoning about object-oriented programs, complicates the development of verified compilers and static analyzers, and limits the potential for principled language design [? ?].

Several attempts have been made to formalize object-oriented concepts. [?] proposed an imperative object calculus, while [?] introduced Featherweight Java as a minimal core calculus. However, these formalisms either focus on specific language features or omit constructs commonly found in modern OO languages. As [?] observed, the development of object-based programming languages has suffered from the lack of any generally accepted formal foundations.

This paper presents φ -calculus, a calculus designed to serve as a formal foundation for object-oriented programming. The calculus is built around a single primitive—the object formation—which encapsulates attributes, data, and functions. Objects interact through application, which attaches expressions to attributes, and dispatch, which retrieves attached expressions.

The contributions of this paper are as follows:

- A formal syntax for φ -calculus defined by a context-free grammar (Section 3).
- Rigorous definitions of the core constructs (Section 4).
- A family of semantic operators (Section 5).
- A reference interpreter that runs φ -expressions².

¹L^AT_EX sources of this paper are maintained in the REPOSITORY public GitHub repository, the rendered version is 0.0.0.

²<https://github.com/objectnary/phino>

The remainder of this paper is organized as follows. Section 2 provides an informal overview of the calculus. Section 3 presents the syntax as a context-free grammar. Section 4 defines the foundational concepts. Section 5 introduces the semantic operators. Section 6 discusses related work. Finally, Section 7 suggests directions of future studies.

2 Informal Overview

An object *formation* is a collection of *attributes*, which are uniquely named pairs, for example:

$$\llbracket \text{price} \mapsto \emptyset, \text{color} \mapsto \llbracket \Delta \mapsto \text{FF-C0-CB} \rrbracket \rrbracket. \quad (1)$$

This formation has two attributes `price` and `color`. The formation is an *abstract formation* because the `price` attribute is *void*, i.e. there is nothing *attached* to it. The `color` attribute is attached to another formation with one Δ -asset, which is attached to *data* (three bytes).

A *full application* of a pair—an attribute and an *expression*—to an abstract formation *results* in a new *closed* formation, for example:

$$\llbracket x \mapsto \emptyset \rrbracket (x \mapsto b) \rightsquigarrow \llbracket x \mapsto b \rrbracket. \quad (2)$$

A *partial application* results in a new abstract formation, for example:

$$\llbracket x \mapsto \emptyset, y \mapsto \emptyset \rrbracket (x \mapsto b) \rightsquigarrow \llbracket x \mapsto b, y \mapsto \emptyset \rrbracket.$$

A formation may be *dispatched* from another formation with the help of *dot notation* where the right side *accesses* the left side, for example:

$$\llbracket x \mapsto \xi.\overline{\text{p}(\text{t} \mapsto b)}.\text{y}, \text{p} \mapsto \llbracket \text{t} \mapsto \emptyset, \text{y} \mapsto \xi.\text{t} \rrbracket \rrbracket.x \rightsquigarrow b. \quad (3)$$

The leftmost symbol ξ denotes the formation itself; the following `.p` part retrieves the formation attached to the attribute `p`; then, the application $(\text{t} \mapsto b)$ makes a copy of the formation; finally, the `.y` part retrieves formation *b*.

Attributes are *immutable*, i.e. an application to a formation, where the attribute is already attached, results in a *terminator* denoted as $\perp$ (error), for example:

$$\llbracket x \mapsto b_1 \rrbracket (x \mapsto b_2) \rightsquigarrow \perp.$$

An expression may be textually reduced, for example:

$$\begin{aligned} & \llbracket x \mapsto \emptyset, y \mapsto \emptyset \rrbracket (x \mapsto b_1)(y \mapsto b_2) \rightsquigarrow \\ & \rightsquigarrow \llbracket x \mapsto b_1, y \mapsto \emptyset \rrbracket (y \mapsto b_2) \rightsquigarrow \\ & \rightsquigarrow \llbracket x \mapsto b_1, y \mapsto b_2 \rrbracket. \end{aligned}$$

The formation on the left is reduced to the formation on the right in two reduction steps. Some expressions may be in *normal form*, which means no further applicable reductions.

A formation is called a *decorator* if it has the φ -attribute with an expression attached to it, known as a *decoratee*. An attribute dispatched from a decorator reduces to the same attribute dispatched from the decoratee, if the attribute is not present in the decorator, for example:

$$\llbracket \varphi \mapsto \llbracket x \mapsto b \rrbracket \rrbracket.x \rightsquigarrow b.$$

$\langle \text{Expression} \rangle$	$\rightarrow \langle \text{Formation} \rangle \mid \langle \text{Application} \rangle \mid \langle \text{Dispatch} \rangle \mid \langle \text{Locator} \rangle \mid \perp$
$\langle \text{Formation} \rangle$	$\rightarrow \llbracket \langle \text{Binding} \rangle \rrbracket$
$\langle \text{Application} \rangle$	$\rightarrow \langle \text{Expression} \rangle (\langle \text{A-Pair} \rangle)$
$\langle \text{A-Pair} \rangle$	$\rightarrow \langle \tau\text{-Pair} \rangle \mid \langle \alpha\text{-Pair} \rangle$
$\langle \text{Dispatch} \rangle$	$\rightarrow \langle \text{Expression} \rangle . \langle \text{Attribute} \rangle$
$\langle \text{Locator} \rangle$	$\rightarrow \Phi \mid \xi$
$\langle \text{Binding} \rangle$	$\rightarrow \langle \text{Pair} \rangle \langle \text{Bindings} \rangle \mid \epsilon$
$\langle \text{Bindings} \rangle$	$\rightarrow , \langle \text{Pair} \rangle \langle \text{Bindings} \rangle \mid \epsilon$
$\langle \text{Pair} \rangle$	$\rightarrow \langle \emptyset\text{-Pair} \rangle \mid \langle \tau\text{-Pair} \rangle \mid \langle \Delta\text{-Pair} \rangle \mid \langle \lambda\text{-Pair} \rangle$
$\langle \emptyset\text{-Pair} \rangle$	$\rightarrow \langle \text{Attribute} \rangle \mapsto \emptyset$
$\langle \tau\text{-Pair} \rangle$	$\rightarrow \langle \text{Attribute} \rangle \mapsto \langle \text{Expression} \rangle$
$\langle \alpha\text{-Pair} \rangle$	$\rightarrow \langle \text{Alpha} \rangle \mapsto \langle \text{Expression} \rangle$
$\langle \Delta\text{-Pair} \rangle$	$\rightarrow \Delta \mapsto \langle \text{Data} \rangle$
$\langle \lambda\text{-Pair} \rangle$	$\rightarrow \lambda \mapsto \langle \text{Function} \rangle$

Fig. 1. Syntax as a context-free grammar, in BNF.

A formation may have data attached only to its Δ -asset, for example:

$$\llbracket x \mapsto \llbracket \Delta \mapsto 00-2A \rrbracket \rrbracket.$$

A formation may have a *function* attached to its λ -asset. Such a formation is referred to as an *atom*. An atom may be *morphed* to another expression by evaluating its function, for example:

$$\langle \llbracket \lambda \mapsto \text{Sqrt}, x \mapsto 256 \rrbracket, e, s \rangle \Rightarrow \langle \llbracket \Delta \mapsto 16 \rrbracket, s \rangle. \quad (4)$$

An expression may be *dataized* by morphing its atoms first and then extracting the data attached to the Δ -asset of the formation, for example:

$$\langle \llbracket \lambda \mapsto \text{Sqrt}, x \mapsto 256 \rrbracket, e, s \rangle \Downarrow \langle 16, s \rangle.$$

3 Syntax

The syntax is defined by BNF in Fig. 1, where the starting symbol is $\langle \text{Expression} \rangle$.

Besides the literals mentioned in the grammar in orange, the alphabet includes three non-terminals that rewrite to terminals as follows:

- $\langle \text{Attribute} \rangle$: either 1) Greek letter φ , 2) Greek letter ρ , or 3) a string of lowercase English letters possibly with dashes inside, e.g. “price” or “a-car”;
- $\langle \text{Data} \rangle$: a sequence of bytes in hexadecimal format, e.g. “EF-41-5C” is a sequence of three bytes, “42-” is a one-byte sequence (with a trailing dash to avoid confusion with integers), and “--” (double dash) is an empty sequence of bytes;
- $\langle \text{Function} \rangle$: a string of English letters and numbers where the first symbol is an uppercase letter, e.g. “Sqrt” or “F1”;
- $\langle \text{Alpha} \rangle$: a Greek letter α with a non-negative whole-number index, e.g. α_2 .

Table 1 shows all possible syntax sugar.

Table 1. Syntax sugar.

Syntax sugar	Its more verbose equivalent
$e(\tau_1 \mapsto e_1, \tau_2 \mapsto e_2, \dots)$	$e(\tau_1 \mapsto e_1)(\tau_2 \mapsto e_2) \dots$
$e(e_0, e_1, \dots)$	$e(\alpha_0 \mapsto e_0, \alpha_1 \mapsto e_1, \dots)$
$\tau_1(\tau_2, \tau_3, \dots) \mapsto \llbracket B \rrbracket$	$\tau_1 \mapsto \llbracket \tau_2 \mapsto \emptyset, \tau_3 \mapsto \emptyset, \dots, B \rrbracket$
$\tau_1 \mapsto \tau_2$	$\tau_1 \mapsto \xi.\tau_2$
$\llbracket B \rrbracket$	$\llbracket B, \rho \mapsto \emptyset \rrbracket$ if $\rho \notin B$
"你好"	$\Phi.\text{string}(\Phi.\text{bytes}(\llbracket \Delta \mapsto \text{E4-BD-A0-E5-A5-BD} \rrbracket))$ (UTF-8 string)
42	$\Phi.\text{number}(\Phi.\text{bytes}(\llbracket \Delta \mapsto 40-45-00-00-00-00-00-00 \rrbracket))$ (eight bytes per integer, assuming it has a floating point)
3.14	$\Phi.\text{number}(\Phi.\text{bytes}(\llbracket \Delta \mapsto 40-09-1E-B8-51-EB-85-1F \rrbracket))$ (eight bytes per number with a floating point)

4 Foundations

In this section, we introduce the core concepts of φ -calculus. The purpose of this section is to define the entities that constitute the calculus—such as expressions, formations, and attributes—together with their structural relationships. The definitions given here are descriptive rather than operational. They explain what expressions are, not how they are evaluated or transformed.

Definition 4.1 (Expression). An **expression**, ranged over $\mathcal{E}$ by e_i , is a grammatical construct obeying the syntax of Fig. 1.

Definition 4.2 (Attribute). An **attribute**, ranged over $\mathcal{T}$ by τ_i , is an identifier to which either $\emptyset$ or an expression may be attached.

Definition 4.3 (Alpha). An **alpha** is a positional identifier.

Definition 4.4 (Void vs. Attached). An attribute is **void** if $\emptyset$ is attached to it, otherwise it is an **attached** attribute.

Definition 4.5 (Data). **Data**, ranged over $\mathcal{D}$ by δ_i , is a possibly empty sequence of 8-bit bytes.

Definition 4.6 (State). A **state** of evaluation s_i ranges over $\mathcal{S}$.

Definition 4.7 (Function). A **function** is a total mapping $\mathcal{B} \times \mathcal{E} \times \mathcal{S} \rightarrow \mathcal{E} \times \mathcal{S}$ that deterministically maps formation, universe, and state to an expression and a new state, ranged over $\mathcal{F}$ by f_i .

Definition 4.8 (Asset). An **asset** is an identifier to which either data (denoted as Δ -asset) or function (denoted as λ -asset) is attached.

Definition 4.9 (Binding). A **binding**, ranged over $\mathcal{G}$ by B_i , is a possibly empty sequence of key-value pairs, where all keys are unique.

Definition 4.10 (Formation). An object **formation**, denoted as $\llbracket B \rrbracket$, ranged over $\mathcal{B}$ by b_i , is a binding.

If $b = \llbracket B \rrbracket$, then the predicate $k \in b$ holds if the key k is present in B .

We introduce the term “formation” rather than using the more traditional term “construction” because the latter generally implies the presence of a class from which an object is being constructed or instantiated. Instead, formation is closer to the creation of a prototype, which may either be used “as is” or copied.

The following is an example of a formation with four pairs, where the first one is an asset attached to data, while the other three are attributes attached to expressions:

$$\llbracket \Delta \mapsto 00-2A, b \mapsto b_2(\alpha_0 \mapsto b_3).bar, a \mapsto \llbracket \lambda \mapsto Sqrt \rrbracket, foo \mapsto \perp \rrbracket. \quad (5)$$

The arrow $\mapsto$ denotes an attachment of an expression (right-hand side) to an attribute (left-hand side). The arrow $\mapsto$ denotes an attachment of data or function to an asset.

Definition 4.11 (Domain). A **domain** of binding B , denoted as $\bar{B}$, is a sequence of all attributes of B , excluding assets.

The domain of the binding in the formation of Eq. (5) is $\langle b, a, foo \rangle$. Assets Δ and λ do not belong to the domain.

Definition 4.12 (Terminator). The **terminator**, denoted as $\perp$, is an expression that signals an error.

Definition 4.13 (Abstract Formation). A formation with at least one void attribute is an **abstract formation**.

Equation (1) is an example of an abstract formation, while the formation attached to its **color** attribute is closed.

A formation that is not an abstract formation may be called a *closed* formation.

Definition 4.14 (Application). An **application**, denoted as $e_1(\tau \mapsto e_2)$ or $e_1(\alpha_i \mapsto e_2)$, means an attempt to attach e_2 (the “argument”) to either the τ attribute of e_1 (the “subject”) or the i -th attribute of e_1 .

Equation (2) demonstrates application, where $a \mapsto b_2$ is applied to an abstract formation $\llbracket a \mapsto \emptyset \rrbracket$. The application creates a new formation $\llbracket a \mapsto b_2 \rrbracket$, while the existing formation remains intact.

The formation in Eq. (1) is an abstract formation, because its attribute **price** is void. The formation in Eq. (2) was an abstract formation before the application, but the formation created by the application is closed since its attribute **a** is not void (attached to b_2).

Even though $\emptyset$ being attached to an attribute of a formation resembles NULL references, there is a significant difference: in φ -calculus, void attributes may be attached to expressions only once, while any further reattachments are prohibited.

In $b(\alpha_i \mapsto e)$ application, e must be attached to the i -th attribute of b .

Definition 4.15 (Dispatch). A **dispatch**, denoted as $e.\tau$, where e is the “subject,” means an attempt to retrieve what is attached to τ in the formation to which e may be transformed.

Definition 4.16 (Atom). An **atom** is a formation with a function attached to its λ -asset.

Equation (4) demonstrates an atom with a function that calculates the square root of a number, which it retrieves from the Δ -asset of $b.\alpha_0$ with the help of the morphing function (Section 5.5). The implementation of functions is outside the scope of φ -calculus: they may be implemented, for example, in λ -calculus or a programming language such as Java or C++.

Definition 4.17 (Decoration). **Decoration** is a mechanism of extending a formation (“decoratee”) by attaching it to the φ -attribute of a formation (“decorator”), which makes attributes of the decoratee retrievable from the decorator, unless the decorator has its own attributes with the same names.

Definition 4.18 (Decorator). A formation is a **decorator** if it has an φ -attribute with an expression attached to it; the expression attached to the φ -attribute is its **decoratee**.

Definition 4.19 (Parent). Attaching expression e to the ρ attribute of formation b means setting the **parent** of b to e .

The presence of “parent” in each formation is essential for the coordination of inner formations after dispatch. Consider the following abstract formation with an inner formation:

$$[[x \mapsto [a \mapsto \emptyset, \text{next} \mapsto [\varphi \mapsto \xi.\rho.a.\text{plus}(1)]]], k \mapsto \xi.x(42)]].$$

Here, if the parent of $\Phi.x.\text{next}$ were attached in the formation, the result of dataization of $\Phi.k.\text{next}$ would not be equal to 43. Instead, it would be equal to $\perp$, because $\Phi.k.\text{next}.\rho$ would still be attached to $\Phi.x$ after the dispatch of $\Phi.k.\text{next}$. The parent attribute may be compared with the **this** pointer in Java or C++, which does not point anywhere until a method of a class is called. Then, when the method is called, the **this** pointer refers to the formation that owns the method.

5 Operators

In this section we introduce a family of semantic operators defined on ϕ -expressions. These operators do not extend the syntax of the calculus. Instead, they describe systematic transformations of expressions that reveal, modify, or project their semantic content. Each operator is defined in terms of the reduction semantics introduced earlier and preserves the core meaning of expressions while possibly changing their form or role.

5.1 Contextualization

Definition 5.1 (Universe). A **universe** is an expression that serves as the global context of another expression, in which the Φ locator refers to the universe.

$$\begin{array}{c}
 \text{R}_{\text{CG}} \frac{}{\langle \Phi, k \rangle \Rightarrow \langle \Phi \rangle} \quad \text{R}_{\text{CXI}} \frac{}{\langle \xi, k \rangle \Rightarrow \langle k \rangle} \quad \text{R}_{\text{CT}} \frac{}{\langle \perp, k \rangle \Rightarrow \langle \perp \rangle} \\
 \\
 \text{R}_{\text{CF}} \frac{}{\langle \llbracket B \rrbracket, k \rangle \Rightarrow \langle \llbracket B \rrbracket \rangle} \quad \text{R}_{\text{CD}} \frac{\langle n, k \rangle \Rightarrow \langle n_1 \rangle}{\langle n.\tau, k \rangle \Rightarrow \langle n_1.\tau \rangle} \\
 \\
 \text{R}_{\text{CA}} \frac{\langle n, k \rangle \Rightarrow \langle n_1 \rangle \quad \langle e_1, k \rangle \Rightarrow \langle n_2 \rangle}{\langle n(\tau \mapsto e_1), k \rangle \Rightarrow \langle n_1(\tau \mapsto n_2) \rangle} \\
 \\
 \text{R}_{\text{CAA}} \frac{\langle n, k \rangle \Rightarrow \langle n_1 \rangle \quad \langle e_1, k \rangle \Rightarrow \langle n_2 \rangle}{\langle n(\alpha_i \mapsto e_1), k \rangle \Rightarrow \langle n_1(\alpha_i \mapsto n_2) \rangle}
 \end{array}$$

Fig. 2. Contextualization by induction.

Definition 5.2 (Absolute). An expression is **absolute**, denoted by k and ranging over $\mathcal{K} \subseteq \mathcal{E}$, if it is either a) Φ , b) formation, c) dispatch with an absolute subject, or d) application with absolute subject and argument.

Definition 5.3 (Contextualization). The *contextualization* function $\mathcal{E} \times \mathcal{K} \rightarrow \mathcal{E}$, denoted as $\langle e_1, k \rangle \Rightarrow \langle e_2 \rangle$ and specified by the rules of Fig. 2, turns an expression e_1 into e_2 by replacing with the absolute k every ξ that denotes the context of e_1 rather than that of a nested formation.

THEOREM 5.4. *The contextualization function is total, meaning that every expression can be contextualized and its derivation terminates.*

5.2 Normalization

An expression that may be rewritten by the *rules* (or *reductions*) listed in Fig. 3 is a *reducible* expression. The notation $e_1 \rightsquigarrow e_2$, optionally followed by a condition, denotes a reduction of e_1 to e_2 , if the condition holds.

A specific reduction may be denoted, for example, as $\rightsquigarrow_{\text{R}_{\text{DOT}}}$, or just $\rightsquigarrow$ when no specific reduction is meant. The notation $e_1 \rightsquigarrow^* e_2$ denotes a reflexive transitive closure of all reductions, so that there is a possibly empty finite sequence of reductions between e_1 and e_2 .

Definition 5.5 (Normal Form). An expression that has no more possible applications of reductions is *irreducible* or a *normal form*, denoted as n_i ranging over $\mathcal{N} \subseteq \mathcal{E}$.

Thus, n is a normal form of e if $e \rightsquigarrow^* n$ and there is no expression e_1 such that $n \rightsquigarrow e_1$.

Not every expression has a normal form. For some expressions normalization may never terminate, for example:

R_{ALPHA} :	$\llbracket B_1, \tau \mapsto \emptyset, B_2 \rrbracket (\alpha_i \mapsto e) \rightsquigarrow \llbracket B_1, \tau \mapsto \emptyset, B_2 \rrbracket (\tau \mapsto e)$ if $i = \overline{B_1} $
R_{AMISS} :	$\llbracket B \rrbracket (\alpha_i \mapsto e) \rightsquigarrow \perp$ if $ \overline{B} \leq i$
R_{COPY} :	$\llbracket B_1, \tau \mapsto \emptyset, B_2 \rrbracket (\tau \mapsto k) \rightsquigarrow \llbracket B_1, \tau \mapsto k, B_2 \rrbracket$
R_{DC} :	$\perp (\tau \mapsto e) \rightsquigarrow \perp$
R_{DCA} :	$\perp (\alpha_i \mapsto e) \rightsquigarrow \perp$
R_{DD} :	$\perp.\tau \rightsquigarrow \perp$
R_{DL} :	$\llbracket B_1, \lambda \mapsto F, B_2 \rrbracket \rightsquigarrow \perp$ if $\Delta \in B_1$ or $\Delta \in B_2$
R_{DOT} :	$\llbracket B_1, \tau \mapsto n, B_2 \rrbracket.\tau \rightsquigarrow e(\rho \mapsto \llbracket B_1, \tau \mapsto n, B_2 \rrbracket)$ where $\langle n, \llbracket B_1, \tau \mapsto n, B_2 \rrbracket \rangle \Rightarrow \langle e \rangle$
R_{MISS} :	$\llbracket B \rrbracket (\tau \mapsto e) \rightsquigarrow \perp$ if $\tau \notin B$
R_{NULL} :	$\llbracket B_1, \tau \mapsto \emptyset, B_2 \rrbracket.\tau \rightsquigarrow \perp$
R_{OVER} :	$\llbracket B_1, \tau \mapsto e_1, B_2 \rrbracket (\tau \mapsto e_2) \rightsquigarrow \perp$ if $\tau \neq \rho$
R_{OVERA} :	$\llbracket B_1, \tau \mapsto e_1, B_2 \rrbracket (\alpha_i \mapsto e_2) \rightsquigarrow \perp$ if $i = \overline{B_1} $ and $\tau \neq \rho$
R_{ϕ} :	$\llbracket B \rrbracket.\tau \rightsquigarrow \llbracket B \rrbracket.\varphi.\tau$ if $\varphi \in B$ and $\tau \notin B$
R_{STAY} :	$\llbracket B_1, \rho \mapsto e_1, B_2 \rrbracket (\rho \mapsto e_2) \rightsquigarrow \llbracket B_1, \rho \mapsto e_1, B_2 \rrbracket$
R_{STOP} :	$\llbracket B \rrbracket.\tau \rightsquigarrow \perp$ if $\tau \notin B$ and $\varphi \notin B$ and $\lambda \notin B$

Fig. 3. Reduction rules, generated by phino 0.0.100.

$$\begin{aligned}
& \frac{\Pi_1}{\llbracket \mathbf{x} \mapsto \mathbf{y}, \mathbf{y} \mapsto \mathbf{x} \rrbracket.\mathbf{x} \rightsquigarrow_{R_{\text{DOT}}}} \\
& \rightsquigarrow \frac{\Pi_2}{\llbracket \Pi_1.\mathbf{y}(\rho \mapsto \Pi_1) \rrbracket \rightsquigarrow_{R_{\text{DOT}}}} \\
& \rightsquigarrow \Pi_1.\mathbf{x}(\rho \mapsto \Pi_1, \rho \mapsto \Pi_1) \rightsquigarrow_{R_{\text{DOT}}} \\
& \rightsquigarrow \frac{\Pi_3}{\llbracket \Pi_2(\rho \mapsto \Pi_1, \rho \mapsto \Pi_1) \rrbracket \rightsquigarrow_{R_{\text{DOT}}}} \\
& \rightsquigarrow \Pi_1.\mathbf{x}(\rho \mapsto \Pi_1, \rho \mapsto \Pi_1, \rho \mapsto \Pi_1, \rho \mapsto \Pi_1) \rightsquigarrow_{R_{\text{DOT}}} \\
& \rightsquigarrow \Pi_3(\rho \mapsto \Pi_1, \rho \mapsto \Pi_1) \rightsquigarrow \\
& \rightsquigarrow \dots
\end{aligned}$$

Section A demonstrates how normalization works through examples.

THEOREM 5.6. *The reduction relation is confluent: if $e \rightsquigarrow^* e_1$ and $e \rightsquigarrow^* e_2$, then there exists an expression e_3 such that $e_1 \rightsquigarrow^* e_3$ and $e_2 \rightsquigarrow^* e_3$.*

Confluence guarantees that the normal form of an expression is unique, irrespective of the order in which the rules are applied.

5.3 Equivalence

Definition 5.7 (Equivalence). Two expressions are said to be syntactically equivalent or just **equivalent** (denoted by $\equiv$) if their normal forms are syntactically identical.

5.4 Evaluation

Definition 5.8 (Evaluation). The *evaluation* function $\mathcal{B} \times \mathcal{E} \times \mathcal{S} \rightarrow \mathcal{E} \times \mathcal{S}$, denoted as $\langle b, e, s_1 \rangle \rightarrow \langle n, s_2 \rangle$, maps an atom b to n , calling by value the function attached to its λ -asset in the universe e , possibly shifting the state s_1 to s_2 :

$$\frac{F(\llbracket B_1, \lambda \mapsto F, B_2 \rrbracket, e, s_1) \rightarrow \langle n, s_2 \rangle}{\langle \llbracket B_1, \lambda \mapsto F, B_2 \rrbracket, e, s_1 \rangle \rightarrow \langle n, s_2 \rangle}$$

5.5 Morphing

Definition 5.9 (Morphing). The *morphing* function $\mathcal{N} \times \mathcal{E} \times \mathcal{S} \rightarrow (\mathcal{B} \cup \{T\}) \times \mathcal{S}$, denoted as $\langle n_1, e, s_1 \rangle \Rightarrow \langle n_2, s_2 \rangle$ and specified by the rules of Fig. 4, reduces a normal form n_1 to the formation it denotes, or to $\perp$ if it denotes none, by evaluating the atoms it reaches and resolving the Φ locator against the universe e , possibly shifting the state s_1 to s_2 . Morphing is a partial function, since it normalizes the expressions it reaches (Section 5.2) and normalization may not terminate.

THEOREM 5.10. *The morphing function is confluent, meaning that every normal form can be morphed to at most one formation or $\perp$.*

5.6 Dataization

Definition 5.11 (Dataization). The *dataization* function $\mathcal{N} \times \mathcal{E} \times \mathcal{S} \rightarrow \mathcal{D} \times \mathcal{S}$, denoted as $\langle n, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle$ and specified by the rules of Fig. 5, extracts the datum δ that a normal form n produces by morphing n to a formation in the universe e and reading the data attached to its Δ -asset, possibly shifting the state s_1 to s_2 .

THEOREM 5.12 (PROGRESS). *Dataization is defined for every normal form except the terminator $\perp$, meaning that for any $n \neq \perp$, e , and s_1 at least one rule of Fig. 5 applies, so a derivation never gets stuck at a term that ought to carry data. The terminator $\perp$ matches no rule: a derivation that reaches it halts without a datum, which is exactly the error that $\perp$ signals.*

THEOREM 5.13. *The dataization function is confluent, meaning that every normal form can be dataized to at most one datum, provided the functions attached to atoms are pure—their result depends on nothing beyond the formation, the universe, and the state.*

Section B demonstrates how the dataization function works through examples.

5.7 Congruence

Definition 5.14 (Congruence). Two expressions n_1 and n_2 are said to be behaviorally equivalent or **congruent** (denoted by $\cong$) if for any state s_1 and any e :

$$\langle n_1, e, s_1 \rangle \Downarrow \langle n, s_2 \rangle \quad \text{and} \quad \langle n_2, e, s_1 \rangle \Downarrow \langle n, s_2 \rangle.$$

$$\begin{array}{c}
R_{MF} \frac{}{\langle \llbracket B \rrbracket, e, s \rangle \Rightarrow \langle \llbracket B \rrbracket, s \rangle} \\
R_{\lambda} \frac{\langle \llbracket B_1, \lambda \mapsto F, B_2 \rrbracket, e, s_1 \rangle \rightarrow \langle n, s_2 \rangle \quad n.\tau \rightsquigarrow n_1 \quad \langle n_1, e, s_2 \rangle \Rightarrow \langle n_2, s_3 \rangle}{\langle \llbracket B_1, \lambda \mapsto F, B_2 \rrbracket.\tau, e, s_1 \rangle \Rightarrow \langle n_2, s_3 \rangle} \\
R_{MD} \frac{\langle n, e, s_1 \rangle \Rightarrow \langle n_1, s_2 \rangle \quad n_1.\tau \rightsquigarrow n_2 \quad \langle n_2, e, s_2 \rangle \Rightarrow \langle n_3, s_3 \rangle}{\langle n.\tau, e, s_1 \rangle \Rightarrow \langle n_3, s_3 \rangle} \text{ if } n \notin \mathcal{B} \\
R_{MA} \frac{\langle n, e, s_1 \rangle \Rightarrow \langle n_1, s_2 \rangle \quad n_1(\tau \mapsto k_1) \rightsquigarrow n_2 \quad \langle n_2, e, s_2 \rangle \Rightarrow \langle n_3, s_3 \rangle}{\langle n(\tau \mapsto k_1), e, s_1 \rangle \Rightarrow \langle n_3, s_3 \rangle} \\
R_{MAA} \frac{\langle n, e, s_1 \rangle \Rightarrow \langle n_1, s_2 \rangle \quad n_1(\alpha_i \mapsto k_1) \rightsquigarrow n_2 \quad \langle n_2, e, s_2 \rangle \Rightarrow \langle n_3, s_3 \rangle}{\langle n(\alpha_i \mapsto k_1), e, s_1 \rangle \Rightarrow \langle n_3, s_3 \rangle} \\
R_{MAD} \frac{\langle \perp, e, s_1 \rangle \Rightarrow \langle n_2, s_2 \rangle}{\langle n(\tau \mapsto n_1), e, s_1 \rangle \Rightarrow \langle n_2, s_2 \rangle} \text{ if } n_1 \notin \mathcal{K} \\
R_{MAAD} \frac{\langle \perp, e, s_1 \rangle \Rightarrow \langle n_2, s_2 \rangle}{\langle n(\alpha_i \mapsto n_1), e, s_1 \rangle \Rightarrow \langle n_2, s_2 \rangle} \text{ if } n_1 \notin \mathcal{K} \\
R_{\Phi} \frac{e \rightsquigarrow n \quad \langle n, e, s_1 \rangle \Rightarrow \langle n_1, s_2 \rangle}{\langle \Phi, e, s_1 \rangle \Rightarrow \langle n_1, s_2 \rangle} \text{ if } e \neq \Phi \quad R_{DEAD} \frac{}{\langle \perp, e, s \rangle \Rightarrow \langle \perp, s \rangle} \\
R_{XI} \frac{\langle \perp, e, s_1 \rangle \Rightarrow \langle n, s_2 \rangle}{\langle \xi, e, s_1 \rangle \Rightarrow \langle n, s_2 \rangle} \quad R_{MG} \frac{\langle \perp, e, s_1 \rangle \Rightarrow \langle n, s_2 \rangle}{\langle \Phi, \Phi, s_1 \rangle \Rightarrow \langle n, s_2 \rangle}
\end{array}$$

Fig. 4. Morphing rules, generated by phino 0.0.100.

$$\begin{array}{c}
R_{\Delta} \frac{}{\langle \llbracket B_1, \Delta \mapsto \delta, B_2 \rrbracket, e, s \rangle \Downarrow \langle \delta, s \rangle} \\
R_{BOX} \frac{\langle e_1, \llbracket B_1, \varphi \mapsto e_1, B_2 \rrbracket \rangle \Rightarrow \langle n \rangle \quad n \rightsquigarrow n_1 \quad \langle n_1, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle}{\langle \llbracket B_1, \varphi \mapsto e_1, B_2 \rrbracket, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle} \text{ if } [\Delta, \lambda] \cap (B_1 \cup B_2) = \emptyset \\
R_{FIRE} \frac{\langle \llbracket B_1, \lambda \mapsto F, B_2 \rrbracket, e, s_1 \rangle \rightarrow \langle n, s_2 \rangle \quad n \rightsquigarrow n_1 \quad \langle n_1, e, s_2 \rangle \Downarrow \langle \delta, s_3 \rangle}{\langle \llbracket B_1, \lambda \mapsto F, B_2 \rrbracket, e, s_1 \rangle \Downarrow \langle \delta, s_3 \rangle} \\
R_{NONE} \frac{\langle \perp, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle}{\langle \llbracket B \rrbracket, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle} \text{ if } [\Delta, \lambda, \varphi] \cap B = \emptyset \\
R_{NORM} \frac{\langle n, e, s_1 \rangle \Rightarrow \langle n_1, s_2 \rangle \quad \langle n_1, e, s_2 \rangle \Downarrow \langle \delta, s_3 \rangle}{\langle n, e, s_1 \rangle \Downarrow \langle \delta, s_3 \rangle} \text{ if } n \notin \mathcal{B} \text{ and } n \neq \perp
\end{array}$$

Fig. 5. Dataization rules, generated by phino 0.0.100.

Two congruent expressions may be non-equivalent, for example:

$$e_1 = \llbracket \text{foo} \mapsto \emptyset, \Delta \mapsto 01-02 \rrbracket \quad e_2 = \llbracket \text{bar} \mapsto \emptyset, \Delta \mapsto 01-02 \rrbracket$$

$$e_1 \cong e_2 \not\Rightarrow e_1 \equiv e_2.$$

6 Related Work

In this section we analyze and categorize prior art related to our work. Neither object-oriented formalism nor pure object-oriented languages are new research topics. However, we identified certain gaps in existing studies that make us believe that our work has novelty.

Attempts were made to formalize OOP and introduce an object calculus similar to the lambda calculus [?] used in functional programming. For example, [?] suggested an imperative calculus of objects, which was extended by [?] to support classes, by [?] to support concurrency and synchronization, and by [?] to support distributed programming.

Earlier, [?] combined OOP and π -calculus in order to introduce object calculus for asynchronous communication, which was further referenced by [?] in their work on object-based design notation.

A few attempts were made to reduce existing OOP languages and formalize what is left. Featherweight Java is the most notable example proposed by [?], which omits almost all features of the full language (including interfaces and even assignment) to obtain a small calculus. Later it was extended by [?] to support nested and multi-threaded transactions. Featherweight Java is used in formal languages such as Obsidian [?] and SJF [?].

Another example is Larch/C++ [?], which is a formal algebraic interface specification language tailored to C++. It allows interfaces of C++ classes and functions to be documented in a way that is unambiguous and concise.

Several attempts to formalize OOP were made by extensions of the most popular formal notations and methods, such as Object-Z [?] and VDM++ [?]. In Object-Z, state and operation schemes are encapsulated into classes. The formal model is based upon the idea of a class history [?]. However, all these OO extensions do not have comprehensive refinement rules that can be used to transform specifications into implemented code in an actual OO programming language, as was noted by [?].

[?] suggested an object calculus as an extension to relational calculus. [?] further developed these ideas and related them to a Boolean algebra. [?] developed an algorithm to transform an object calculus into an object algebra.

However, all these theoretical attempts to formalize OO languages were unable to fully describe their features, as was noted by [?]: “The development of concurrent object-based programming languages has suffered from the lack of any generally accepted formal foundations for defining their semantics.” In addition, when describing the attempts of formalization, [?] summarized: “Not one of the notations is defined formally, nor provided with denotational semantics, nor founded on axiomatic semantics.” Moreover, despite these efforts, [?] noted in their series of works that

relatively little formal work has been carried out on object-based languages, and it remains true to this day.

7 Future Studies

This paper intentionally focuses on defining φ -calculus (syntax, foundations, and semantic operators) and does not yet demonstrate practical use. At the same time, it motivates the calculus as a step toward more rigorous reasoning, verified compilers and static analyzers, and better language design. Future work may build on φ -calculus by creating the following artifacts:

- A short list of core laws about reductions (what always works, what may fail).
- A clear way to prove that two expressions behave the same in any context.
- A type system that catches mistakes early.
- A way to mark which expressions are pure and which may have side effects.
- A catalog of safe rewrites (how to change code without changing meaning).
- A translation from Java or C++ into φ -expressions.
- A standard library of built-in atoms with precise rules.
- Larger language features built as add-ons (overriding, interfaces, constructors).
- Add-ons for mutation, concurrency, and distribution, with simple safety guarantees.

We invite readers to extend, formalize, and apply φ -calculus to real object-oriented patterns and languages, and to help turn these foundations into a shared body of results and artifacts.

8 Acknowledgments

Many thanks to (in alphabetical order of last names) Aliaksei Bialiauski, Fabricio Cabral, Kirill Chernyavskiy, Piotr Chmielowski, Danilo Danko, Konstantin Gukov, Andrew Graur, Ali-Sultan Kigizbaev, Nikolai Kudasov, Alexander Legalov, Mikhail Lipanin, Tymur Aysenko, Alexandr Naumchev, Alonso A. Ortega, John Page, Alex Panov, Maxim Petrov, Alexander Pushkarev, Marcos Douglas B. Santos, Alex Semenyuk, Violetta Sim, Sergei Skliar, Stian Soiland-Reyes, Viacheslav Tradunskyi, Ilya Trub, César Soto Valero, Alena Vasileva, David West, Vladimir Zakharov, and Evgeny Zouev for their contribution to the development of φ -calculus.

A Examples of Normalization

The following examples demonstrate how the reduction rules of Fig. 3 may normalize some expressions, involving the contextualization function (Section 5.1):

$$\begin{aligned}
 e_1 &= \overline{\Pi_4} \llbracket \mathbf{x} \mapsto \mathbf{t}, \mathbf{t} \mapsto \emptyset \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow \Pi_4 . \mathbf{t}(\rho \mapsto \Pi_4) \rightsquigarrow_{\mathbf{R}_{\text{NULL}}} \\
 &\rightsquigarrow \perp(\rho \mapsto \Pi_4) \rightsquigarrow_{\mathbf{R}_{\text{DC}}} \\
 &\rightsquigarrow \perp. \\
 \\
 e_2 &= \llbracket \mathbf{x} \mapsto \overline{\Pi_5} \llbracket \mathbf{t} \mapsto 42 \rrbracket . \mathbf{t} \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow 42. \\
 \\
 e_3 &= \llbracket \mathbf{x} \mapsto \emptyset \rrbracket (\alpha_1 \mapsto 42) . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{ALPHA}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto \emptyset \rrbracket (\rho \mapsto 42) . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto \emptyset, \rho \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{NULL}}} \\
 &\rightsquigarrow \perp. \\
 \\
 e_4 &= \llbracket \mathbf{x} \mapsto \emptyset \rrbracket (42) . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{ALPHA}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto \emptyset \rrbracket (\mathbf{x} \mapsto 42) . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow 42. \\
 \\
 e_5 &= \llbracket \mathbf{x} \mapsto \llbracket \lambda \mapsto \text{Fn} \rrbracket . \rho . \mathbf{k}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{NULL}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto \perp . \mathbf{k}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DD}}} \\
 &\rightsquigarrow \overline{\Pi_6} \llbracket \mathbf{x} \mapsto \perp, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow \perp(\rho \mapsto \Pi_6) \rightsquigarrow_{\mathbf{R}_{\text{DC}}} \\
 &\rightsquigarrow \perp. \\
 \\
 e_6 &= \llbracket \mathbf{x} \mapsto \emptyset, \mathbf{y} \mapsto \mathbf{x} \rrbracket (\mathbf{x} \mapsto 42) . \mathbf{y} \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
 &\rightsquigarrow \overline{\Pi_7} \llbracket \mathbf{x} \mapsto 42, \mathbf{y} \mapsto \mathbf{x} \rrbracket . \mathbf{y} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow \Pi_7 . \mathbf{x}(\rho \mapsto \Pi_7) \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
 &\rightsquigarrow 42. \\
 \\
 e_7 &= \llbracket \mathbf{x} \mapsto \mathbf{t}, \mathbf{t} \mapsto \emptyset \rrbracket (\mathbf{t} \mapsto 42) \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
 &\rightsquigarrow \llbracket \mathbf{x} \mapsto \mathbf{t}, \mathbf{t} \mapsto 42 \rrbracket .
 \end{aligned}$$

$$\begin{aligned}
e_8 &= \overline{\Pi_8} \llbracket \mathbf{x} \mapsto \mathbf{t}, \varphi \mapsto \llbracket \mathbf{t} \mapsto \llbracket \rrbracket \rrbracket \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_8 . \mathbf{t}(\rho \mapsto \Pi_8) \rightsquigarrow_{\mathbf{R}_\phi} \\
&\rightsquigarrow \Pi_8 . \varphi . \mathbf{t}(\rho \mapsto \Pi_8) \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{t} \mapsto \llbracket \rrbracket \rrbracket (\rho \mapsto \Pi_8) . \mathbf{t}(\rho \mapsto \Pi_8) \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
&\rightsquigarrow \overline{\Pi_9} \llbracket \mathbf{t} \mapsto \llbracket \rrbracket, \rho \mapsto \Pi_8 \rrbracket . \mathbf{t}(\rho \mapsto \Pi_8) \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \rrbracket (\rho \mapsto \Pi_9, \rho \mapsto \Pi_8) \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
&\rightsquigarrow \llbracket \rho \mapsto \Pi_9 \rrbracket (\rho \mapsto \Pi_8) \rightsquigarrow_{\mathbf{R}_{\text{STAY}}} \\
&\rightsquigarrow \llbracket \rho \mapsto \Pi_9 \rrbracket . \\
e_9 &= \overline{\Pi_{10}} \llbracket \mathbf{x} \mapsto \rho . \rho . \mathbf{t} \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{10} . \rho . \rho . \mathbf{t}(\rho \mapsto \Pi_{10}) \rightsquigarrow_{\mathbf{R}_{\text{NULL}}} \\
&\rightsquigarrow \perp . \rho . \mathbf{t}(\rho \mapsto \Pi_{10}) \rightsquigarrow_{\mathbf{R}_{\text{DD}}} \\
&\rightsquigarrow \perp . \mathbf{t}(\rho \mapsto \Pi_{10}) \rightsquigarrow_{\mathbf{R}_{\text{DD}}} \\
&\rightsquigarrow \perp (\rho \mapsto \Pi_{10}) \rightsquigarrow_{\mathbf{R}_{\text{DC}}} \\
&\rightsquigarrow \perp . \\
e_{10} &= \llbracket \mathbf{x} \mapsto \llbracket \rrbracket . \rho . \mathbf{k}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{NULL}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \perp . \mathbf{k}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DD}}} \\
&\rightsquigarrow \overline{\Pi_{11}} \llbracket \mathbf{x} \mapsto \perp, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \perp (\rho \mapsto \Pi_{11}) \rightsquigarrow_{\mathbf{R}_{\text{DC}}} \\
&\rightsquigarrow \perp . \\
e_{11} &= \overline{\Pi_{12}} \llbracket \mathbf{x} \mapsto \mathbf{t}(\mathbf{k} \mapsto \mathbf{f}) . \mathbf{k}, \mathbf{t}(\mathbf{k}) \mapsto \llbracket \rrbracket, \mathbf{f} \mapsto \llbracket \rrbracket \rrbracket . \mathbf{x} \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{12} . \mathbf{t}(\mathbf{k} \mapsto \Pi_{12} . \mathbf{f}) . \mathbf{k}(\rho \mapsto \Pi_{12}) \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{k} \mapsto \emptyset \rrbracket (\rho \mapsto \Pi_{12}, \mathbf{k} \mapsto \llbracket \rrbracket (\rho \mapsto \Pi_{12})) . \mathbf{k}(\rho \mapsto \Pi_{12}) \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
&\rightsquigarrow \llbracket \mathbf{k} \mapsto \emptyset, \rho \mapsto \Pi_{12} \rrbracket (\mathbf{k} \mapsto \overline{\Pi_{13}} \llbracket \rho \mapsto \Pi_{12} \rrbracket) . \mathbf{k}(\rho \mapsto \Pi_{12}) \rightsquigarrow_{\mathbf{R}_{\text{COPY}}} \\
&\rightsquigarrow \llbracket \mathbf{k} \mapsto \Pi_{13}, \rho \mapsto \Pi_{12} \rrbracket . \mathbf{k}(\rho \mapsto \Pi_{12}) \rightsquigarrow_{\mathbf{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{13}(\rho \mapsto \llbracket \mathbf{k} \mapsto \Pi_{13}, \rho \mapsto \Pi_{12} \rrbracket, \rho \mapsto \Pi_{12}) \rightsquigarrow_{\mathbf{R}_{\text{STAY}}} \\
&\rightsquigarrow \Pi_{13}(\rho \mapsto \Pi_{12}) \rightsquigarrow_{\mathbf{R}_{\text{STAY}}} \\
&\rightsquigarrow \Pi_{13} .
\end{aligned}$$

$$\begin{aligned}
e_{12} &= \overline{\llbracket \mathbf{a} \mapsto \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \emptyset \rrbracket (\mathbf{t} \mapsto \mathbf{k}), \mathbf{k} \mapsto \llbracket \lambda \mapsto \text{Fn} \rrbracket \rrbracket . \mathbf{x} . \mathbf{t} . \mathbf{p} \rrbracket} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \llbracket \mathbf{t} \mapsto \emptyset \rrbracket (\mathbf{t} \mapsto \Pi_{14} . \mathbf{k}, \rho \mapsto \Pi_{14}) . \mathbf{t} . \mathbf{p} \rrbracket \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \llbracket \mathbf{t} \mapsto \emptyset \rrbracket (\mathbf{t} \mapsto \llbracket \lambda \mapsto \text{Fn} \rrbracket (\rho \mapsto \Pi_{14}), \rho \mapsto \Pi_{14}) . \mathbf{t} . \mathbf{p} \rrbracket \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \llbracket \mathbf{t} \mapsto \emptyset \rrbracket (\mathbf{t} \mapsto \overline{\llbracket \lambda \mapsto \text{Fn}, \rho \mapsto \Pi_{14} \rrbracket}, \rho \mapsto \Pi_{14}) . \mathbf{t} . \mathbf{p} \rrbracket \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \llbracket \mathbf{t} \mapsto \Pi_{15} \rrbracket (\rho \mapsto \Pi_{14}) . \mathbf{t} . \mathbf{p} \rrbracket \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \overline{\llbracket \mathbf{t} \mapsto \Pi_{15}, \rho \mapsto \Pi_{14} \rrbracket} . \mathbf{t} . \mathbf{p} \rrbracket \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \Pi_{15} (\rho \mapsto \Pi_{16}) . \mathbf{p} \rrbracket \rightsquigarrow_{\text{R}_{\text{STAY}}} \\
&\rightsquigarrow \llbracket \mathbf{a} \mapsto \Pi_{15} . \mathbf{p} \rrbracket . \\
e_{13} &= \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \llbracket \mathbf{p} \mapsto \rho . \rho . \mathbf{k} \rrbracket . \mathbf{p} \rrbracket . \mathbf{t}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \llbracket \mathbf{p} \mapsto \rho . \rho . \mathbf{k} \rrbracket . \rho . \rho . \mathbf{k} (\rho \mapsto \llbracket \mathbf{p} \mapsto \rho . \rho . \mathbf{k} \rrbracket) \rrbracket . \mathbf{t}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{NULL}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \perp . \rho . \mathbf{k} (\rho \mapsto \llbracket \mathbf{p} \mapsto \rho . \rho . \mathbf{k} \rrbracket) \rrbracket . \mathbf{t}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DD}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \perp . \mathbf{k} (\rho \mapsto \llbracket \mathbf{p} \mapsto \rho . \rho . \mathbf{k} \rrbracket) \rrbracket . \mathbf{t}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DD}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \perp (\rho \mapsto \llbracket \mathbf{p} \mapsto \rho . \rho . \mathbf{k} \rrbracket) \rrbracket . \mathbf{t}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DC}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \perp \rrbracket . \mathbf{t}, \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \mathbf{x} \mapsto \perp (\rho \mapsto \llbracket \mathbf{t} \mapsto \perp \rrbracket) \rrbracket . \mathbf{k} \mapsto 42 \rrbracket . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DC}}} \\
&\rightsquigarrow \overline{\llbracket \mathbf{x} \mapsto \perp, \mathbf{k} \mapsto 42 \rrbracket} . \mathbf{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \perp (\rho \mapsto \Pi_{17}) \rightsquigarrow_{\text{R}_{\text{DC}}} \\
&\rightsquigarrow \perp . \\
e_{14} &= \overline{\llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \rho . \mathbf{k} . \rho . \mathbf{t} \rrbracket, \mathbf{k} \mapsto \llbracket \rrbracket, \mathbf{t} \mapsto \llbracket \rrbracket \rrbracket . \mathbf{x} . \mathbf{t} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \overline{\llbracket \mathbf{t} \mapsto \rho . \mathbf{k} . \rho . \mathbf{t} \rrbracket (\rho \mapsto \Pi_{18}) . \mathbf{t} \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \overline{\llbracket \mathbf{t} \mapsto \Pi_{19}, \rho \mapsto \Pi_{18} \rrbracket} . \mathbf{t} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{20} . \rho . \mathbf{k} . \rho . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{18} (\rho \mapsto \Pi_{20}) . \mathbf{k} . \rho . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \overline{\llbracket \mathbf{x} \mapsto \llbracket \mathbf{t} \mapsto \Pi_{19} \rrbracket, \mathbf{k} \mapsto \llbracket \rrbracket, \mathbf{t} \mapsto \llbracket \rrbracket, \rho \mapsto \Pi_{20} \rrbracket} . \mathbf{k} . \rho . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \rrbracket (\rho \mapsto \Pi_{21}) . \rho . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \overline{\llbracket \rho \mapsto \Pi_{21} \rrbracket} . \rho . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{21} (\rho \mapsto \Pi_{22}) . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{STAY}}} \\
&\rightsquigarrow \Pi_{21} . \mathbf{t} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \llbracket \rrbracket (\rho \mapsto \Pi_{21}, \rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
&\rightsquigarrow \Pi_{22} (\rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{STAY}}} \\
&\rightsquigarrow \Pi_{22} .
\end{aligned}$$

The following expressions may not be reduced any further; they are in normal form:

$$e_{15} = \llbracket \mathbf{x} \mapsto \emptyset \rrbracket (\mathbf{x} \mapsto \mathbf{t}) .$$

$$e_{16} = \llbracket x \mapsto k, k \mapsto 42 \rrbracket.$$

$$e_{17} = \llbracket x \mapsto t, \lambda \mapsto \mathbf{Fn} \rrbracket.$$

$$e_{18} = \llbracket x \mapsto k, t \mapsto 42 \rrbracket.$$

$$e_{19} = \llbracket k \mapsto \llbracket x \mapsto 42, \lambda \mapsto \mathbf{Fn} \rrbracket.y \rrbracket.$$

$$e_{20} = \llbracket x \mapsto \llbracket t \mapsto \Phi.x \rrbracket \rrbracket.$$

B Example of Dataization

Consider an algorithm that converts degrees from Celsius to Fahrenheit:

$$^{\circ}F = ^{\circ}C \times 1.8 + 32,$$

Assuming that $^{\circ}C$ already equals 25 and is attached to the `c` attribute instead of being provided by a user, the algorithm may be represented by an expression:

$$\begin{aligned} & \llbracket \\ & \quad \varphi \mapsto \llbracket \varphi \mapsto \text{c.times}(1.8).\text{plus}(32), \text{c} \mapsto 25 \rrbracket, \\ & \quad \text{bytes}(\text{data}) \mapsto \llbracket \varphi \mapsto \text{data} \rrbracket, \\ & \quad \text{number}(\text{as-bytes}) \mapsto \llbracket \\ & \quad \quad \varphi \mapsto \text{as-bytes}, \\ & \quad \quad \text{times}(\text{x}) \mapsto \llbracket \lambda \mapsto F1 \rrbracket, \\ & \quad \quad \text{plus}(\text{x}) \mapsto \llbracket \lambda \mapsto F2 \rrbracket \\ & \quad \rrbracket \\ & \rrbracket. \end{aligned} \tag{6}$$

The functions are defined as follows:

$$\begin{aligned} R_{F1} & \frac{\langle b.\rho, e, s \rangle \Downarrow \langle \delta_1, s \rangle \quad \langle b.x, e, s \rangle \Downarrow \langle \delta_2, s \rangle}{\langle b, e, s \rangle \rightarrow \langle e.\text{number}(e.\text{bytes}(\text{data} \mapsto \llbracket \Delta \mapsto \delta \rrbracket)), s \rangle} \text{ if } \delta \models \delta_1 \times \delta_2 \\ R_{F2} & \frac{\langle b.\rho, e, s \rangle \Downarrow \langle \delta_1, s \rangle \quad \langle b.x, e, s \rangle \Downarrow \langle \delta_2, s \rangle}{\langle b, e, s \rangle \rightarrow \langle e.\text{number}(e.\text{bytes}(\text{data} \mapsto \llbracket \Delta \mapsto \delta \rrbracket)), s \rangle} \text{ if } \delta \models \delta_1 + \delta_2 \end{aligned}$$

The following sequence of transformations constitutes the dataization of Eq. (6):

C Proofs

This appendix collects the proofs of the theorems stated in Section 5.

PROOF OF THEOREM 5.4. Fix an absolute k and argue by structural induction on the expression e , establishing at once that a derivation of $\langle e, k \rangle \Rightarrow \langle e_1 \rangle$ exists for some e_1 and that it is finite. By the grammar of Fig. 1 an expression is a locator (Φ or ξ), a terminator $\perp$, a formation, a dispatch, or an application, and its outermost constructor tells these cases apart. The rules of Fig. 2 match the cases one-to-one, so exactly one rule applies to every expression. For Φ , ξ , $\perp$ and a formation $\llbracket B \rrbracket$ the matching rule— R_{CG} , R_{CXI} , R_{CT} or R_{CF} —is an axiom that yields a result in a single step, so the base cases hold; R_{CF} rewrites the formation to itself and never descends into B , halting the recursion at every formation boundary. A dispatch $e_1.\tau$ selects R_{CD} , whose single premise contextualizes the subject e_1 ; an application $e_1(\tau \mapsto e_2)$ or $e_1(\alpha_i \mapsto e_2)$ selects R_{CA} or R_{CAA} , whose two premises contextualize e_1 and e_2 . Every premise is a strict subexpression of its conclusion, so by the induction hypothesis each has a finite derivation and a result, and the chosen rule assembles them into a finite derivation of the whole; the argument k is copied wholesale at R_{CXI} and never traversed. The recursion therefore follows the finite syntax tree and every rule is defined on its case, so contextualization is total on $\mathcal{E} \times \mathcal{K}$ and each derivation terminates. $\square$

PROOF OF THEOREM 5.6. We have proved this property mechanically in Lean 4, with the proof available in a public GitHub repository³. $\square$

PROOF OF THEOREM 5.10. **We still need to prove this.** $\square$

PROOF OF THEOREM 5.12. By the grammar of Fig. 1, every expression, and hence every normal form, is a formation, an application, a dispatch, a locator, or the terminator $\perp$; neither the universe e nor the state s_1 restricts which rule fires. We show that every case other than $\perp$ matches a rule, while $\perp$ matches none.

If $n = \perp$, no rule of Fig. 5 applies, since every rule demands a formation or, for R_{NORM} , an expression distinct from $\perp$. The derivation halts with no datum, which is the error we intend: $\perp$ carries no data to read.

If n is neither a formation nor $\perp$, then $n \notin \mathcal{B}$ and $n \neq \perp$, which is exactly the condition of R_{NORM} ; the rule morphs n and dataizes the result, and since morphing delivers a formation or $\perp$ (Section 5.5), that result falls into one of the other cases.

If $n = \llbracket B \rrbracket$ is a formation, we inspect its assets and attributes in order: a) if B carries a Δ -asset, R_{Δ} fires; b) otherwise, if B carries an λ -asset, R_{FIRE} fires; c) otherwise, if B carries an φ -attribute, R_{BOX} fires, its condition $[\Delta, \lambda] \cap B = \emptyset$ granted by the two exclusions above; d) otherwise B holds none of Δ , λ , and φ , which is the condition of R_{NONE} ; that rule fires and reduces the goal to the dataization of $\perp$, which then halts as above — a formation with nothing to dataize is itself an error. So every formation matches a rule, and $\perp$ alone is left without one.

³<https://github.com/objectnary/proof>

Termination is by induction on the derivation. R_Δ is an axiom and discharges with no premise. Each remaining rule reduces the dataization of n to the dataization of a normal form produced by an operator that itself terminates—contextualization (Section 5.1) and normalization (Section 5.2) for R_{BOX} , evaluation and normalization for R_{FIRE} , and morphing (Section 5.5) for R_{NORM} , while R_{NONE} reduces directly to $\perp$ —so every premise is again a dataization judgment, and R_{NORM} never recurs into itself. The derivation is hence finite whenever the recursion through R_{BOX} and R_{FIRE} is well-founded, which holds for every expression that denotes a terminating computation; such a derivation ends either at a datum or at the terminator $\perp$. $\square$

PROOF OF THEOREM 5.13. Dataization is specified by the syntax-directed inference system of Fig. 5, so confluence amounts to the relation $\Downarrow$ being single-valued: whenever $\langle n, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle$ and $\langle n, e, s_1 \rangle \Downarrow \langle \delta', s'_2 \rangle$, we have $\delta = \delta'$ and $s_2 = s'_2$. We argue by induction on the first derivation, after observing that at most one rule applies to any configuration $\langle n, e, s_1 \rangle$, so both derivations are headed by the same rule.

The conclusions of the five rules partition every configuration, and $\perp$ falls under none of them. If $n = \perp$, no rule matches, so dataization is undefined and there is nothing to disprove. If n is neither a formation nor $\perp$, only R_{NORM} matches. If $n = \llbracket B \rrbracket$ is a formation, the asset and attribute it carries select a single rule: a) a Δ -asset selects R_Δ and an λ -asset selects R_{FIRE} , the two being mutually exclusive in a normal form; b) with neither asset present, an φ -attribute selects R_{BOX} and its absence selects R_{NONE} , the latter pair separated by the explicit conditions $[\Delta, \lambda] \cap B = \emptyset$ and $[\Delta, \lambda, \varphi] \cap B = \emptyset$. Hence no configuration matches two rules.

Because the same rule heads both derivations, it suffices to check that each rule maps its premises to a single result. R_Δ is an axiom whose conclusion reads the result directly off n —the attached data—leaving the state untouched, so it admits no other outcome. R_{NONE} reduces to the dataization of $\perp$, which is undefined, so it yields no result and hence no second one. R_{BOX} chains three premises: contextualization is a total function (Section 5.1) and yields a unique n ; normalization is confluent (Section 5.2) and yields a unique normal form n_1 ; and the dataization of n_1 is unique by the induction hypothesis. R_{NORM} chains morphing, which is confluent (Section 5.5) and yields a unique formation or $\perp$, with a dataization premise that is unique by the induction hypothesis. R_{FIRE} evaluates the function F attached to the λ -asset. By Section 5.4 a function is a total mapping that deterministically maps formation, universe, and state to an expression and a new state; being pure, its result depends on nothing else. The universe e is fixed throughout the derivation and the state propagates only through the explicit threading of the rules, so both derivations present the call with identical arguments and it returns the same $\langle n, s_2 \rangle$; normalization then yields a unique n_1 and the dataization premise is unique by the induction hypothesis. In every case the preceding premises are determined before the recursive one, so the sub-derivations dataize the same normal form in the same state and the induction hypothesis applies. The two derivations therefore agree on the result, and dataization relates each normal form to at most one data and state. $\square$